DXF 自动房间识别与 SVG 热区标注方案

版本: v1.0

日期: 2026-04-28

涉及模块: M1 资产与空间可视化

---

1. 背景与目标

Admin 管理后台支持将楼栋 DXF 文件上传至后端，后端调用 `split_dxf_by_floor.py` 将其切分为每个楼层的独立 SVG 文件。但切分后的 SVG 仅包含 CAD 线稿层（ `<g id="floor-plan">` ），热区层（ `<g id="unit-hotspots">` ）始终为空占位符，无法在 Web 端呈现可交互的房间色块。

本方案目标：**自动化提取 DXF 中的房间轮廓多边形，填充 SVG 热区层，使每个可租房间成为可点击、带状态色块的交互单元。**

---

2. DXF 数据分析结论

经对 `A座.dxf` （全楼 260 万行）进行图层与实体类型分析，核心发现如下：

2.1 可用图层清单

图层名	实体类型	数量估计	用途
`0-面积框线`	闭合 **LWPOLYLINE** + TEXT	全楼 300+ 个多边形	每个多边形精确对应一个计量面积单元
`0-面积框线填充`	HATCH/SOLID	与面积框对应	配套填充，验证边界用
房号	CIRCLE + TEXT（01/02/03…）	每层 10 ~ 30 个	房间顺序编号
房间名称	TEXT	每层若干	房间功能名（走廊、储藏室等）
`SPACE`	ACAD*PROXY*ENTITY（天正）	全楼若干	不可读，私有格式，不使用

2.2 坐标系分析

ezdxf SVGBackend 输出的 SVG viewBox 坐标系与 DXF Model Space 坐标系存在以下映射关系：

$$
svg\x = *dxf*\x - extmin_x
$$

$$
svg\y = *extmax*\y - dxf_y \quad \text{（Y 轴翻转）}
$$

其中 `extmin` / `extmax` 为该楼层区域内所有实体的实际包围盒，存储于每个楼层的 `.json` 元数据文件中的 `dxf_region` 字段（本方案新增）。

---

3. 技术方案

3.1 整体流水线

```
DXF 文件
  │
  ├─ [现有] split_dxf_by_floor.py
  │      → 每楼层 SVG（CAD 线稿）+ JSON 骨架（新增 dxf_region 字段）
  │
  └─ [新增] annotate_hotzone.py
        → 读取 0-面积框线 LWPOLYLINE（房间轮廓）
        → 关联 房号/房间名称 TEXT 实体
        → 坐标变换 → SVG 多边形
        → 写入 SVG unit-hotspots 层
        → 更新 JSON units[] 数组
```

3.2 关键算法

Step 1：房间多边形提取

```
# 过滤条件：图层=0-面积框线 + 实体类型=LWPOLYLINE + closed=True
# 包含条件：多边形质心在该楼层 dxf_region 内
# 过滤小图形：面积 < 1000 sq_units（去除图例框线等干扰）
```

Step 2：关联房号（Shapely 空间包含检测）

```
# 对每个 房号 图层的 CIRCLE 实体，取圆心坐标
# 通过 Polygon.contains(Point(cx, cy)) 确定归属房间
# 对应 TEXT 实体（同一位置）提取编号文字
```

Step 3：关联房间名称

```
# 对每个 房间名称 图层的 TEXT 实体，取插入点坐标
# 通过 Polygon.contains(Point(ix, iy)) 确定归属房间
```

Step 4：坐标变换

```
svg_x = dxf_x - dxf_region["min_x"]
svg_y = dxf_region["max_y"] - dxf_y  # Y 轴翻转
```

Step 5：写入 SVG

```
<!-- unit-hotspots 层 -->
<polygon
```

```
  points="x1,y1 x2,y2 ..."
  data-unit-id="01"
  data-room-name="办公室"
  data-area-sqm="125.6"
  data-area-type="全面积"
  class="unit-vacant"                <!-- 默认空置，前端运行时根据租务状态切换 -->
/>
```

Step 6：更新 JSON

```
{
  "units": [
    {
      "room_no": "01",
      "room_name": "办公室",
      "area_sqm": 125.6,
      "area_type": "全面积",
      "svg_points": "x1,y1 x2,y2 ..."
    }
  ]
}
```

---

4. 实施计划

Phase 1 — 本地 Python 脚本（本次实施）

序号	工作内容	文件
1.1	修改 `_inject_hotzone_spec`，在 JSON 骨架中新增 `dxf_region` 字段	`scripts/split_dxf_by_floor.py`
1.2	新建房间识别与热区标注脚本	`scripts/annotate_hotzone.py`
1.3	集成到流水线脚本	`scripts/build_floors.sh`

Phase 2 — 后端 API（后续排期）

序号	工作内容	文件
2.1	新增 `PATCH /api/floors/:id/hotspots` 端点	`backend/lib/modules/assets/controllers/floor_plan_controller.dart`
2.2	切分完成后自动调用 `annotate_hotzone.py`，写入 DB	`backend/lib/modules/assets/services/cad_import_service.dart`

Phase 3 — Admin 前端消费（后续排期）

序号	工作内容	文件
3.1	Admin 楼层平面图页面渲染 unit-hotspots 层	`admin/src/views/assets/FloorPlanView.vue`
3.2	点击房间弹出详情/跳转 Unit 详情页	同上

---

5. 依赖

依赖	版本要求	安装方式	说明
`ezdxf`	≥1.1	已在 `.venv`	DXF 读取
`shapely`	≥2.0	`pip install shapely`	多边形点包含检测
`lxml`	≥4.9	已在 `.venv`	SVG XML 操作

检查依赖：

```
source .venv/bin/activate
python3 -c "import shapely; print('shapely', shapely.__version__)"
```

---

6. 使用方式

方式一：完整流水线（推荐）

```
bash scripts/build_floors.sh building_a
```

自动执行：DXF → SVG 切分 → 热区标注（→ 可选 PNG）

方式二：仅重新标注热区

```
source .venv/bin/activate
python3 scripts/annotate_hotzone.py \
    cad_intermediate/building_a/A座.dxf \
    cad_intermediate/building_a/floors \
    --prefix A座
```

方式三：仅处理指定楼层

```
python3 scripts/annotate_hotzone.py \
    cad_intermediate/building_a/A座.dxf \
```

```
    cad_intermediate/building_a/floors \
    --prefix A座 --floor F11
```

---

7. 验证方式

```
# 1. 检查 JSON units 已填充
python3 -c "
import json
d = json.load(open('cad_intermediate/building_a/floors/A座_F11.json'))
print(f'房间数: {len(d[\"units\"])}')
print(d['units'][:2])
"

# 2. 检查 SVG 热区层已填充
grep -c 'data-unit-id' cad_intermediate/building_a/floors/A座_F11.svg

# 3. 浏览器预览（在项目根目录启动 HTTP Server）
python3 -m http.server 8765
# 打开 http://localhost:8765/floor_preview.html
# 选择 A座_F11，应看到黄色/红色半透明多边形覆盖在房间轮廓上
```

---

8. 注意事项

1. **坐标变换精度**：`dxf_region` 存储的是实体实际包围盒（`actual_bb`），而非 TK 图框范围，两者可能有 0.1% 的微小偏差，不影响视觉对齐。
1. **多层共用平面图**：`A座_F6-F8-F10.svg` 一个 SVG 对应多个实际楼层，会提取该图框内所有房间。后续后端绑定 `floor_id` 时需人工确认。
1. **弧线处理**：LWPOLYLINE 的 `bulge` 字段不为零时表示弧段。当前脚本用线段近似（每段弧最多 8 个插值点），适用于绝大多数矩形/多边形房间。
1. **面积计算**：面积由 Shapely 从多边形顶点自动推算，单位与 DXF 坐标单位一致（通常为 mm），输出到 JSON 时除以 1,000,000 转换为 m^2。
1. **`unit_id` 绑定**：当前 `data-unit-id` 使用的是 `房号` TEXT 编号（01/02…）或自动递增序号，后续 Phase 2 后端接口上线后需替换为数据库 `units.id`。